# Agent Postmortem Generator

## Part 5 - Postmortem Generation Engine

### Module Design Document

**One-line purpose:**

*Transform analysis results and canonical trace evidence into a deterministic, blameless, evidence-bound postmortem report.*

**Project:** Agent Postmortem Generator
**Module:** Postmortem Generation Engine
**Scope:** MVP design for local-first deterministic report assembly

## 1. Executive Overview

The Postmortem Generation Engine converts the output of the Analysis Engine into a human-readable incident report. It does not inspect raw framework traces, and it does not attempt to rediscover causality. Its job is to take deterministic findings, evidence references, and trace metadata, then assemble them into a stable postmortem structure that can be rendered downstream.

This module exists because analysis and communication are different problems. The Analysis Engine answers what happened and why it happened. The Postmortem Generation Engine answers how to present that answer without inventing facts, changing the meaning, or leaking framework-specific structure into the final document.

In the system pipeline, this module sits immediately after the Analysis Engine and immediately before the Output Layer. The module owns report composition and report wording, while the Output Layer owns file serialization, formatting, and delivery targets such as Markdown files, JSON exports, or future PDF generation.

```
Pipeline position:

Input -> Parser -> Validation -> Normalizer -> Canonical Trace Model
                                            -> Analysis Engine
                                            -> Postmortem Generation Engine
                                            -> Output Layer
```

## 2. Problem Statement

Failed AI agent runs are difficult to explain cleanly because the failure signal is usually spread across many events: tool calls, retries, aborted branches, missing outputs, exception wrappers, and partial state transitions. The raw analysis result can be accurate and still be unusable if the final report is inconsistent, too verbose, too vague, or quietly embeds unsupported claims.

This module solves the report-assembly problem. It turns structured findings into a stable narrative with explicit evidence links, fixed section ordering, and deterministic phrasing. The hard part is not writing prose. The hard part is writing prose that never exceeds the evidence, never depends on hidden judgment calls, and remains readable when the trace is incomplete or messy.

The MVP needs this module because the product value is the postmortem itself. Without a disciplined generation layer, the tool would stop at analysis artifacts and never deliver the artifact engineers actually need in practice: a coherent incident summary that can be read, shared, reviewed, and used as the starting point for remediation.

## 3. Goals

- Produce a deterministic postmortem document from a validated analysis package and canonical evidence references.
- Preserve the exact meaning of upstream analysis without re-ranking, re-inferring, or embellishing it.
- Keep the report structure stable so fixtures, regression tests, and downstream renderers can rely on it.
- Make missing or ambiguous information visible instead of masking it with filler text.
- Bind every substantive claim to either a canonical evidence reference or an explicitly labeled analysis conclusion.
- Emit a render-ready report representation that is independent of the final file format.

Success is not measured by clever wording. Success is measured by consistency, traceability, and the absence of invented facts.

## 4. Non-Goals

- Reconstructing causality from raw trace events. That belongs to the Analysis Engine.
- Parsing or validating framework-specific trace formats. That belongs to earlier layers.

- Producing PDF, HTML, or other final files. That belongs to the Output Layer.
- Using an LLM to rewrite, summarize, or 'improve' the report text.
- Generating remediation tickets, assigning owners, or integrating with issue trackers.
- Comparing multiple runs or detecting drift across runs.
- Providing interactive editing, collaboration, or comment workflows.
- Supporting framework-specific vocabulary once the trace has already been normalized.

Later versions may add alternative report layouts or export targets, but the MVP should remain a single deterministic report path.

# 5. Responsibilities

## What this module owns

- Selecting the report template and section order.
- Transforming analysis findings into report sections and blocks.
- Formatting evidence citations and trace references into report-friendly form.
- Applying deterministic wording rules and blameless tone constraints.
- Emitting a structured report object plus a Markdown representation.
- Performing consistency checks that compare claims against evidence references.

## What this module must never own

- Raw trace parsing, normalization, or schema validation.
- Causal inference beyond the conclusions already produced by analysis.
- File system writes, storage lifecycle, or output path management.
- LLM prompting, self-correction loops, or probabilistic report generation.
- Product opinions such as incident severity policy or remediation prioritization strategy.
- Framework-specific formatting logic that would break framework agnosticism after normalization.

# 6. Inputs

The module accepts an analysis package produced from the canonical trace model. The package is intentionally narrow: it should contain only the facts needed to generate a postmortem, plus the evidence pointers required to justify those facts.

| Input | Required | What it contains | Validation rule | Malformed handling |
|---|---|---|---|---|
| Analysis package | Yes | Ordered findings, causal chain, summary, diagnostics, confidence labels, evidence references | Must be schema-valid and reference only known evidence ids | Reject with a deterministic input error if the schema is invalid; otherwise continue with warnings for partial content |
| Canonical evidence bundle | Yes | Resolved trace snippets, timestamps, node ids, event order metadata, correlation ids | All referenced items must exist and be readable | Unknown references become explicit 'evidence unavailable' markers in the report |
| Report configuration | Optional | Report title, generation | Configuration values must use the | Unsupported settings are ignored |

| | | | | |
|---|---|---|---|---|
| | | timestamp, output variant, verbosity level | supported enum or scalar types | or rejected based on strictness mode |

Assumptions:

- The Analysis Engine has already established the first meaningful fault or the closest available equivalent when the trace is incomplete.
- Evidence references are stable identifiers, not free-form text.
- The canonical trace model has already removed framework-specific structure that should not leak into the report.
- The module is allowed to represent unknowns explicitly, but not to infer new facts.

Malformed input handling follows a simple rule: fail hard on structural invalidity, degrade gracefully on missing evidence, and never fabricate a claim to keep a section looking full.

If the analysis package is syntactically valid but semantically incomplete, the generator still produces a report with clear gaps marked as unresolved or unavailable. That is a feature, not a bug, because silence is often more misleading than a visible omission.

## 7. Outputs

The module does not write files. It emits a report package that the Output Layer can serialize into one or more formats.

| Artifact | Format | Purpose | Notes |
|---|---|---|---|
| PostmortemReport | Structured in-memory object | Primary render-agnostic representation | Contains metadata, sections, blocks, claims, and evidence links |
| Markdown report body | String | Human-readable default output | Stable section ordering and deterministic bullet ordering are required |
| Report diagnostics | Structured list | Explains missing evidence, contradictions, or incomplete analysis | Diagnostics must be explicit rather than silently dropped |
| Optional machine-readable export | JSON-compatible object | Future downstream integration and tests | Should mirror the report object, not re-encode raw trace internals |

Human-readable artifacts are limited to the report itself: executive summary, failure chain, primary fault, contributing factors, evidence section, and recommended follow-ups. The module should not generate decorative prose beyond what the template requires.

## 8. Internal Design

The module is easiest to reason about if it is split into a small set of internal subcomponents with hard boundaries. These are logical components, not necessarily separate packages, but they should behave as if they were isolated.

| Subcomponent | Responsibility | Boundary |
|---|---|---|
| Report Context Builder | Collects metadata, trace identity, analysis summary, and report settings into a normalized context | Never reads raw traces directly |
| Section Planner | Chooses the fixed section list and decides which items belong in each section | Never invents new findings |
| Narrative Assembler | Converts facts into paragraphs, bullets, and short tables | Never changes the meaning of a finding |
| Evidence Binder | Attaches evidence ids and trace references to each substantive claim | Never creates evidence that was not provided |
| Consistency Guard | Checks for unsupported claims, missing citations, duplicate facts, and section drift | Never rewrites the report to hide issues |

Workflow:

```
1. Receive the analysis package and report configuration.
2. Build a normalized report context.
3. Map analysis findings into a fixed report template.
4. Assemble section content using deterministic wording rules.
5. Bind evidence references to claims and narrative bullets.
6. Run consistency checks over the full report object.
7. Emit the final report object and Markdown representation.
```

Important internal decisions:

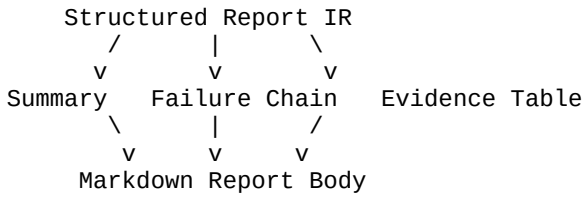
- Section order is fixed. The report should not rearrange sections based on incident shape.
- The generator should not decide the root cause. It only communicates the root cause chosen upstream.
- If a section has no data, it still exists and states that the information was not available.
- The module should prefer short, direct sentences over elaborate prose.
- If analysis gives multiple candidate causes, the generator preserves the ordering provided by analysis rather than inventing its own ranking.

Component view:

```
Analysis Package
   |
   v
Report Context Builder -> Section Planner -> Narrative Assembler -> Evidence Binder ->
Consistency Guard
                                                                          |
                                                                          v
                                                                PostmortemReport IR
                                                                          |
                                                                          v
                                                                Markdown Render
```

Data flow view:

```
Canonical Trace Evidence + Analysis Conclusions
                |
                v
         Report Context
                |
                v
```




## 9. Data Model

The report model should be intentionally boring. Boring is good here. A stable intermediate representation makes the output layer simpler and makes tests much more reliable.

| Entity | Key fields | Why it exists |
|---|---|---|
| PostmortemReport | report_id, metadata, sections, diagnostics, version | Top-level container for the complete report |
| ReportMetadata | project_name, module_part, generated_at, trace_id, run_id, analysis_version | Captures provenance and keeps the report auditable |
| ReportSection | section_id, title, order, blocks | Represents one fixed part of the report template |
| ContentBlock | type, text, bullets, rows, evidence_refs | Keeps sections render-agnostic and testable |
| FindingSummary | finding_id, label, description, confidence, evidence_refs, status | Represents the primary conclusions from analysis |
| CausalChainStep | step_id, sequence, cause, effect, evidence_refs | Preserves the failure chain in ordered form |
| EvidenceCitation | evidence_id, trace_node_id, event_index, snippet, note | Connects a report claim back to the canonical trace model |
| ActionItem | item_id, priority, recommendation, rationale, evidence_refs | Captures recommended next steps without assigning operational ownership |
| DiagnosticEntry | code, severity, message, related_refs | Makes ambiguity and missing data visible to users and tests |

Relationships:

- A PostmortemReport contains many ReportSections.
- A ReportSection contains many ContentBlocks.
- A ContentBlock can reference zero or more EvidenceCitations.
- A FindingSummary can point to many EvidenceCitations but should never be presented without a source reference unless it is explicitly marked as unresolved analysis output.
- A DiagnosticEntry can be attached at the report level or to a specific section.

Why this model exists:

- It keeps the generator independent from the final file format.
- It gives the test suite a stable object graph to snapshot and assert against.
- It prevents prose generation from collapsing into a pile of ad hoc string concatenation.

- It allows later renderers to produce Markdown, HTML, or PDF without changing the core generation logic.

## 10. Algorithms / Processing Logic

The logic must be deterministic, straightforward, and nearly free of hidden heuristics. The module is not an inference engine. It is a report compiler.

Step-by-step processing:

1. Validate that the analysis package and evidence bundle satisfy the schema contract.
2. Build a canonical ordering of findings, causal steps, and action items using the upstream order as the primary key and stable tie-breakers only when needed.
3. Populate the fixed report template from the normalized context.
4. Generate section text using approved wording patterns that preserve the upstream meaning.
5. Attach evidence citations to every substantive claim that can be traced to a canonical event or analysis statement.
6. Insert explicit placeholders such as 'not available from trace' where the evidence is incomplete.
7. Run a final consistency pass to detect orphan claims, missing evidence, duplicate content, or section violations.
8. Emit the PostmortemReport object and a Markdown body that mirrors it exactly.

Edge cases and decision rules:

- If the trace ends before the failure is fully observable, the report should say that the failure is only partially observed.
- If there are multiple plausible causes, preserve the ordered list from analysis and label unresolved ambiguity clearly.
- If timestamps are missing, do not invent chronology from semantic guesswork. Use event order only.
- If evidence snippets overlap or repeat, deduplicate them deterministically by evidence id and event index.
- If the analysis package contains unsupported free-form explanations, include them only when they are already marked as analysis conclusions and never as new claims.
- If a section would otherwise be empty, keep the section and state why it is empty.

Determinism requirements:

- Sort by explicit order fields first, then by timestamp, then by event index, then by stable identifier string.
- Never depend on hash map iteration order for report content.
- Never use randomness, time-based phrasing, or model-generated paraphrasing.
- Never change wording based on hidden thresholds unless those thresholds are configured and tested.

## 11. Tradeoffs

| Axis | Chosen design | Why this is the right tradeoff |
|---|---|---|
| Simplicity vs flexibility | Fixed report template with a small set of configurable presentation knobs | A stable template is easier to test, easier to review, and less likely to drift into chaos |
| Generality vs specificity | Optimized for AI agent failure reporting, not generic incident management | Specificity keeps the wording and section structure relevant to the product's core value |
| Determinism vs heuristic inference | Almost no heuristics inside the generator | The product's credibility depends on not inventing facts at the presentation layer |

| | | |
|---|---|---|
| Compactness vs completeness | Prefer concise sections with explicit gaps over long narrative filler | Long reports are only useful if they remain evidence-driven and scannable |

The preferred design is intentionally conservative. A more flexible generator would be easier to customize, but it would also be harder to trust, harder to test, and more likely to absorb analysis logic that belongs elsewhere.

## 12. Risks and Failure Modes

| Risk | Failure mode | Mitigation |
|---|---|---|
| Upstream ambiguity | Analysis does not provide a single definitive fault | Preserve ambiguity explicitly and label unresolved findings |
| Evidence mismatch | A claim points to a missing or stale evidence reference | Fail validation for hard errors and emit diagnostics for partial mismatch |
| Template creep | New sections appear for every edge case | Keep one canonical template and require a design review for any new section |
| Tone drift | Text becomes blamey, speculative, or inconsistent | Use deterministic wording rules and a small, reviewed phrasing library |
| Hidden dependence on trace shape | Framework-specific quirks leak into report language | Treat the canonical trace model as the only source of truth and reject framework labels at this layer |
| Maintenance drift | Report wording changes without test updates | Use golden tests for both the report IR and Markdown output |

Hidden complexity:

- Causality can look simple after analysis has already done the hard work, but the report still has to present branching, retries, missing nodes, and partial observability in a way that does not overclaim.
- The module can also become a dumping ground for 'nice to have' prose additions. That is exactly how deterministic systems become inconsistent over time.

## 13. Testing Strategy

| Test layer | What it covers | Example cases | Pass criteria |
|---|---|---|---|
| Unit tests | Section assembly, wording rules, evidence binding, sorting | Single-fault run, missing evidence, multiple candidate causes | Outputs are deterministic and claim-evidence aligned |
| Integration tests | Analysis package to report package flow | Full happy-path trace, trace with retries, trace with partial failure | Generated report matches expected structure and meaning |
| Fixture-driven tests | Known trace scenarios stored as fixtures | Tool failure, missing timestamp, concurrent | Golden reports remain stable across runs |

| | | | |
|---|---|---|---|
| | | branch, no-root-cause case | |
| Regression tests | Language drift and section drift | Any change to template, wording, or ordering | Diffs are intentional, reviewed, and justified |

Testing principles:

- Every substantive section should have a fixture that exercises both the happy path and at least one failure or missing-data path.
- Markdown snapshots should be treated as contract tests, not as cosmetic artifacts.
- Report IR tests are more important than rendered file tests because they catch semantic drift before formatting obscures it.
- No test should require a model call or remote dependency.

## 14. Implementation Checklist

9. Define the PostmortemReport intermediate representation before writing any string-rendering logic.
10. Lock the canonical section order and the block types supported by each section.
11. Implement the report context builder and the evidence binder before the Markdown renderer.
12. Add consistency checks for missing citations, unresolved claims, duplicated evidence, and empty sections.
13. Create fixtures for at least one clean success case, one partial trace, one multi-branch retry case, and one ambiguous root-cause case.
14. Write golden tests for the report object and the Markdown output.
15. Reject any attempt to add framework-specific text at this layer.
16. Keep the entrypoint narrow so the Output Layer can consume one clear report package.

A good build order is: model first, template second, binding third, rendering fourth, tests fifth. Anything else invites ad hoc string logic and later regret.

## 15. Future Expansion

- Alternative presentation styles from the same report IR, such as a shorter executive variant or a more verbose engineering variant.
- PDF or HTML rendering as a separate output concern, without changing the generation core.
- Export adapters for issue trackers or incident management tools.
- Multi-run comparison sections that summarize drift or recurrence patterns.
- Localized templates if the project eventually needs non-English reporting.
- Controlled phrasing variants for organizations that want different incident vocabulary, provided they stay evidence-bound.

These are natural extensions because they build on the same stable report IR. They do not require the generator to start doing more analysis or more inference.

## 16. Final Recommendation

This design is MVP-ready as long as the module stays template-driven, evidence-bound, and deliberately boring. That is not a limitation. That is the point. The postmortem generator should be trustworthy before it is configurable.

The only simplification still worth enforcing is to keep one canonical report template for v1. Do not add per-framework report variants, per-team branding logic, or a second phrasing system until the first version has proven stable in the wild.

If the module is built this way, it will support the rest of the system instead of competing with it, and the project will feel like infrastructure rather than a fancy writing demo.

## Implementation note

```
This module should behave like a report compiler:
analysis facts in,
evidence-bound report out,
no hidden judgment calls in between.
```